

Evaluation Comments Template

General Information

- **Evaluator Name:** P1 _____
 - **Application Name:** jpetstore _____
 - **Refactoring Approach:** monomorph _____
-

1) Services Boundaries and Decompositions

1. Are all of the classes in the application mapped to their correct microservices based on their decomposition?

All classes are correctly mapped.

2. Are there classes that were not included in the decomposition but found in the monolith? How were they handled by the approach?

Describe any missing or unaccounted classes:

All classes were included in the decomposition.

3. Are there new classes added to the microservices? Do they still respect the single responsibility of these microservices?

The approach added DTOs and other boilerplate code for ID-based calls as well as protobuf-generated objects. The added classes still respect the single responsibility principle.

4. Has the dependency mapping between components been accurately preserved?

Mostly. Some of the generated imports seem incoherent, e.g.

“org.mybatis.jpetstore.monomorph.registry.ServiceRegistry” is unresolvable and used instead of “org.mybatis.jpetstore.monomorph.id.generated.helpers.ServiceRegistry” which is both used and functional in other generated classes.

5. How well did the approach handle this aspect? (Score from 1 to 10)

9/10.

6. Additional Comments

Couple incorrect references, but easy to fix and breaking until fix.

2) Correctness & Functional Equivalence:

1. Does the refactored application produce the same outcomes as the original monolith?

No. Human intervention would be needed to fix transformation issues.

2. Have any functional behaviors been inadvertently altered during the transformation?

While it looks like functional behaviors have not been altered. The added complexity and requirement for human intervention to fix issues beforehand makes it hard to guarantee.

3. Is the transformation strictly limited to modifying call mechanics, leaving the business logic unchanged?

Yes. Generated DTOs and remote calls are transparent from the business logic point of view.

4. Do the transformed interactions maintain the same logical flow as in the monolith?

All original calls are synchronous so the logical flow remains mostly unaffected by the transformation.

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10.

6. Additional Comments

3) Code Transformation and Remote Invocations:

1. Are all relevant local method calls successfully replaced by their remote call counterparts?

Local calls were replaced by remote calls for the most part, although some issues due to non-replaced imports and broken code generation prevented successful replacement of all calls. For instance, the “order” service’s “AccountActionBean” does not extend “ActionBean” and is missing the expected “Account” type import.

2. Are the request and response structures consistent with the original local calls?

For the most part, yes. Granted the various issues with the transformed services are fixed first.

3. Are there any issues such as missing object mappings or incorrect handling of requests and responses?

Outside of the already mentioned issues mainly regarding the imports, no. The approach also seems to handle correctly the complex Item-Cart-CartItem relationship between the “order” and “catalog” services.

4. Is data correctly serialized and deserialized during remote calls?

Yes. Mappers are used to convert to and from DTOs.

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10.

6. Additional Comments

4) Data Handling and Consistency:

1. Is there an existing database in the monolith? How did the refactoring approach handle it?

The monolith uses MyBatis, a first-class objects persistence framework that which stores SQL queries in XML files that are injected into the corresponding “Mapper” interfaces at runtime. Since the approach correctly duplicate the whole repository, all XML files were copied into all services.

2. How is a shared state managed or synchronized across the services?

3. Is there any data duplication? Do you think that it is justified?

The XML resources holding the resources were duplicated in all services instead of following their corresponding mapper object. This is unjustified and should probably be cleaned up. Although data itself is correctly accessed and not duplicated.

4. Is the data still consistent? Were there any new consistency mechanisms added if needed?

No new mechanisms added.

5. How well did the approach handle this aspect? (Score from 1 to 10)

9/10.

6. Additional Comments

Only some clean up required (which is to be expected from a full copy of the monolith's directory).

5) Performance Overhead:

1. Are there new mechanisms introduced that add overhead to existing interactions?

Yes. The approach adds an ID-based approach as well as DTOs, based on gRPC. The generated DTO clients also encapsulate its own remote calls when inter-services communication is necessary.

2. Do you think that this overhead is justified?

Yes. The added complexity still adds a significant learning curve and lacks transparency compared to the original code in my opinion, but it is still an acceptable solution due to the complexity of the decomposition for this application.

3. What mechanisms would you replace if you had to modify the code manually?

One change I might look into is to hide the gRPC fields and calls from the generated clients and servers so it's easier to compare them to the originals and edit them if needed post-transformation.

4. Are there any indications of over-segmentation leading to excessive inter-service communication?

No. The level of segmentation is coherent with the provided decomposition.

5. How well did the approach handle this aspect? (Score from 1 to 10)

8/10.

6. Additional Comments

6) Code Maintainability & Readability Post-Transformation:

1. Is the refactored code easy to understand and follow compared to the original monolith?

Refactored services are easy to read for the most part. Original code is kept as is. Once again, given the amount of generated classes, using the exact same name as the original class from the monolith was confusing at times.

2. Has the transformation introduced excessive boilerplate code or complexity?

Not excessive. I think the added complexity was legitimate considering the decomposition's complexity.

3. How difficult is it to replace the mechanisms introduced by the approach if needed?

Difficult. Although replacing the mechanisms would be a faster than rewriting the whole thing granted the developers took the time to familiarize themselves with said mechanisms beforehand.

4. Does the approach include fault tolerance or error handling mechanisms?

Some minimal error handling is present through checks such as "response.hasItem". The application already used Spring's "@Transactional" attribute to wrap key functions in transactional context, but I doubt the approach identified it.

5. How well did the approach handle this aspect? (Score from 1 to 10)

7/10.

6. Additional Comments

7) Execution and Packaging:

1. Does the refactored version compile and/or package correctly?

No. The transformed code contains multiple issues such as missing imports that prevents compilation and generated broken classes.

2. Does the refactoring approach introduce new dependencies?

Yes. It adds gRPC and protobuf dependencies, as well as a caching library.

3. Does the approach introduce any new deployment automation mechanisms?

No. It also added the new dependencies to the existing maven project.

4. How well did the approach handle this aspect? (Score from 1 to 10)

7/10.

5. Additional Comments

8) Additional comments that do not fall into any aspect:
